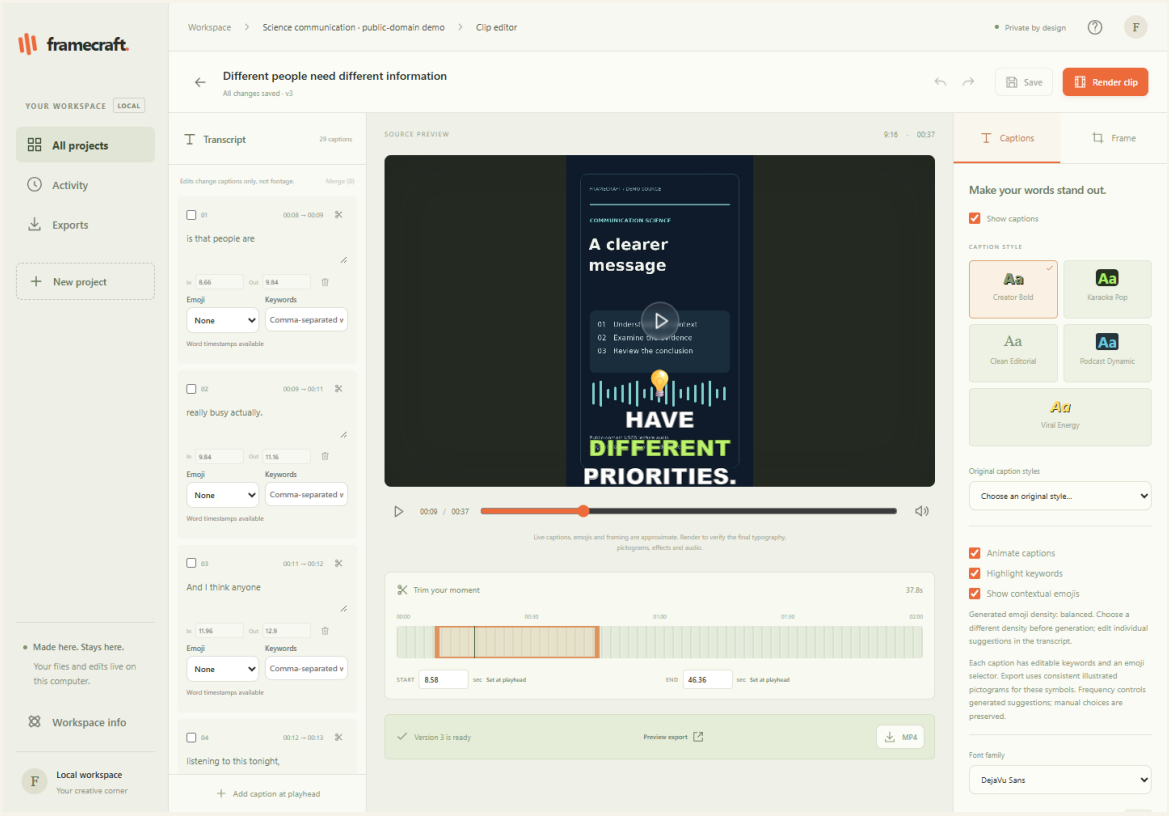
Building an AI video clipping platform

A working local application for finding candidate moments, refining the edit and exporting real short-form video.



Real application capture with isolated, approved demonstration content

290

Automated tests plus four subtests Rerun 29 October 2026

60 min

Continuous source processed Release evidence, 23 October

1080p

Real H.264/AAC MP4 Source-quality limits disclosed

Stage: working local prototype. Python / FastAPI / React / TypeScript / SQLite / Whisper / OpenCV / FFmpeg. Project: github.com/KevBrad/framecraft-ai

The product problem starts with context

A sentence can sound interesting and still fail as a short clip when its question, explanation or payoff is missing. The product must also preserve the creator’s final edit.

01 Selection

Locate a real source interval, retain the surrounding answer and expose why the candidate ranks.

02 Presentation

Correct recognized text and timing, choose emphasis and framing, then inspect the encoded result.

03 Reliability

Handle long recordings, interruption and media errors without overwriting source files or losing edits.

A source-grounded story check

Question + initial answer + later contrast + payoff. A final development comparison retained the complete budget answer and the distinction between a video and a television production. This is a calibration example, not independent editorial-quality validation.

One local app, explicit processing boundaries

01 **Authorized intake**

providers / import service / source library

02 **Normalized, bounded audio**

pipeline / 16 kHz PCM / saved cores

03 **Timestamped speech**

faster-whisper / model profiles

04 **Evidence-based candidates**

extractive topic map / editorial heuristics

05 **Creator-owned edit**

captions / director / persistent versions

06 **Real rendered file**

FFmpeg + libass / ffprobe / ZIP

React/TypeScript and FastAPI form the control plane. Separate durable import and analysis consumers use SQLite WAL schema 2 and local files. No distributed cloud queue or hosted inference service is implied.

Current module map: docs/ARCHITECTURE.md. Saved state covers projects, sources, imports, jobs, edits and versioned exports.

A memory failure changed the processing boundary

243 MiB

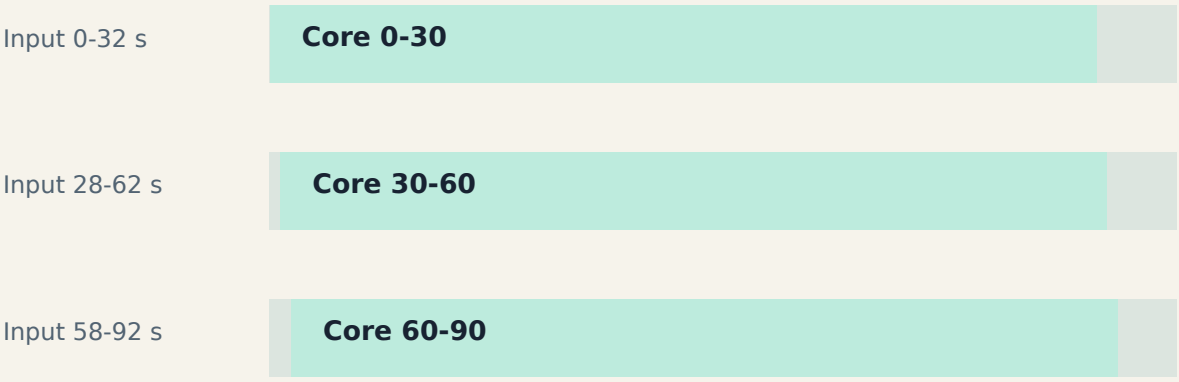
One reported failed complex128 allocation

30 seconds

Recognition core, plus two seconds of context per side

Saved release evidence records an array shaped (1, 79376, 201) and a previous whole-WAV path into speech recognition. Large time-frequency intermediates were consistent with duration-dependent pressure. The exact internal allocation path remains an inference; the old library stack was not re-instrumented for this publication.

Bound the input, then save each completed core



Word midpoint ownership avoids duplicates. Atomic checkpoints retain completed work. The gated Windows worker keeps its 2,048 MiB budget. In 23 October tests, 60 minutes of speech finished analysis in 311.62 seconds with 1,397.46 MiB peak sampled private memory.

Current code and bounded-window tests verify the mechanism. Historical 60-minute processing completed within the retained budget. No upstream-library defect or universal memory bound is claimed.

Validation covers files, workflows and saved state

Current local regression 29 Oct 2026

290 tests + 4 subtests, no skips

Current publication checks 29 Oct 2026

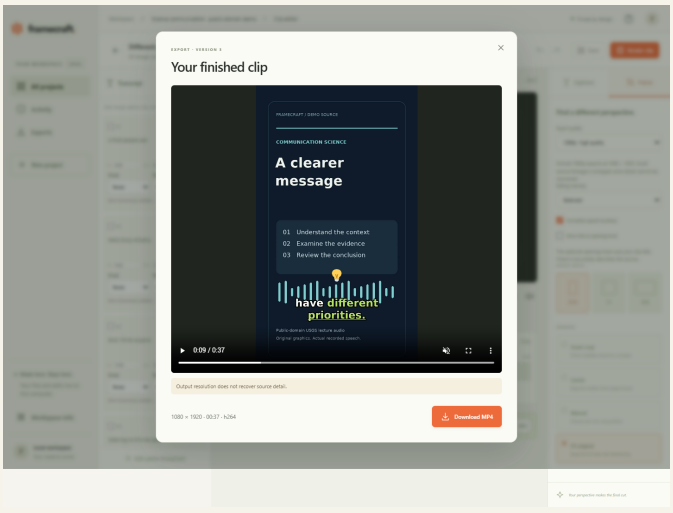
177 lightweight tests + frontend build

Historical browser workflows 23 Oct 2026

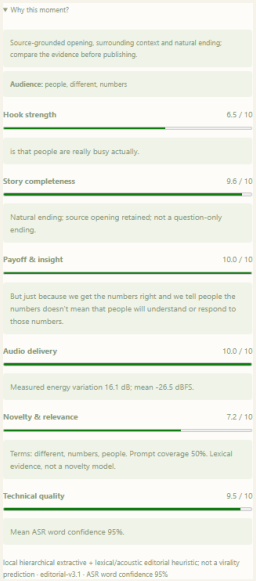
9 new + 11 retained real checks

Historical continuous sources 23 Oct 2026

13 min 29 s / 15 min / 60 min



Actual encoded export preview



Actual ranking evidence

The current audit preserved all database rows and retained original-file hashes. Two long-source exports re-probed as 1080 x 1920 H.264/AAC at 30 fps. No mock response substitutes for an encoded file.

These are engineering results, not users, revenue or market traction. Full test scope, historical timing/memory conditions and limitations: docs/VERIFIED_RESULTS.md.

Product ownership and engineering tradeoffs

What the project demonstrates

Product direction translated into testable workflows. The implementation connects media intake, bounded AI work, persistent editing and verifiable files. Iteration used actual failure cases rather than a compilation result alone.

How AI assistance contributed

KevBrad led requirements, research priorities and iterative acceptance testing. Codex materially assisted code creation, debugging and test authoring.

Tradeoffs and next evidence gates

Local CPU processing and heuristics keep the application usable without paid inference, but recognition and semantic selection remain imperfect. Reliable speaker attribution and forced alignment are incomplete. The next steps are independently labelled highlight/timing data, creator workflow trials and deployment/security review before a hosted pilot.

Project and professional profile

github.com/KevBrad/framecraft-ai

github.com/KevBrad

Seeking product mentorship, creator introductions and independent technical review. Product validation and hosted operational readiness remain future work.